

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) Screenshot... 133747.png

Screenshot 2026-08-07 133747.png(image/png) - 142757 bytes, last modified: 8/7/2026 - 100% done
 Saving Screenshot 2026-08-07 133747.png to Screenshot 2026-08-07 133747.png

```
# End-to-End Vision System Experiment

import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import time

# -----
# 1. Load Images
# -----

# Using the single uploaded image for both, for demonstration purposes.
# If you intend to use two different images, please upload 'image1.jpg' and 'image2.jpg'.
img1 = cv2.imread("Screenshot 2026-08-07 133747.png")
img2 = cv2.imread("Screenshot 2026-08-07 133747.png") # Using the same image for img2

if img1 is None or img2 is None:
    print("Error: Images not found! Please ensure the image files exist and the paths are correct.")
else:
    # Convert to grayscale
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# -----
# 2. Preprocessing
# -----
```

```
start_time = time.time()

blur1 = cv2.GaussianBlur(gray1, (5, 5), 0)
blur2 = cv2.GaussianBlur(gray2, (5, 5), 0)

# -----
# 3. Feature Detection using ORB
# -----

orb = cv2.ORB_create(nfeatures=1000)

kp1, des1 = orb.detectAndCompute(blur1, None)
kp2, des2 = orb.detectAndCompute(blur2, None)

print("Features in Image 1:", len(kp1))
print("Features in Image 2:", len(kp2))

# Draw detected features
feature_img1 = cv2.drawKeypoints(
    gray1, kp1, None,
    color=(0, 255, 0),
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

feature_img2 = cv2.drawKeypoints(
    gray2, kp2, None,
    color=(0, 255, 0),
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# -----
# 4. Feature Matching
# -----

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

matches = bf.match(des1, des2)
```

```
# Sort matches according to distance
matches = sorted(matches, key=lambda x: x.distance)

print("Total Matches:", len(matches))

# Select best 50 matches
best_matches = matches[:50]

matched_image = cv2.drawMatches(
    gray1, kp1,
    gray2, kp2,
    best_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# -----
# 5. PCA Dimensionality Reduction
# -----

# Use descriptors from Image 1
data = des1.astype(np.float32)

# Standardize feature descriptors
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# Reduce dimensions to 2
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)

print("Original Descriptor Dimensions:", data.shape[1])
print("Reduced Dimensions:", data_pca.shape[1])

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

```
print("Total Variance Preserved:",
      sum(pca.explained_variance_ratio_))

# -----
# 6. PCA Visualization
# -----

plt.figure(figsize=(7, 5))

plt.scatter(
    data_pca[:, 0],
    data_pca[:, 1],
    s=20
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization of ORB Features")
plt.show()

# -----
# 7. Display Processing Results
# -----

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(feature_img1)
plt.title("Detected Features - Image 1")
plt.axis("off")

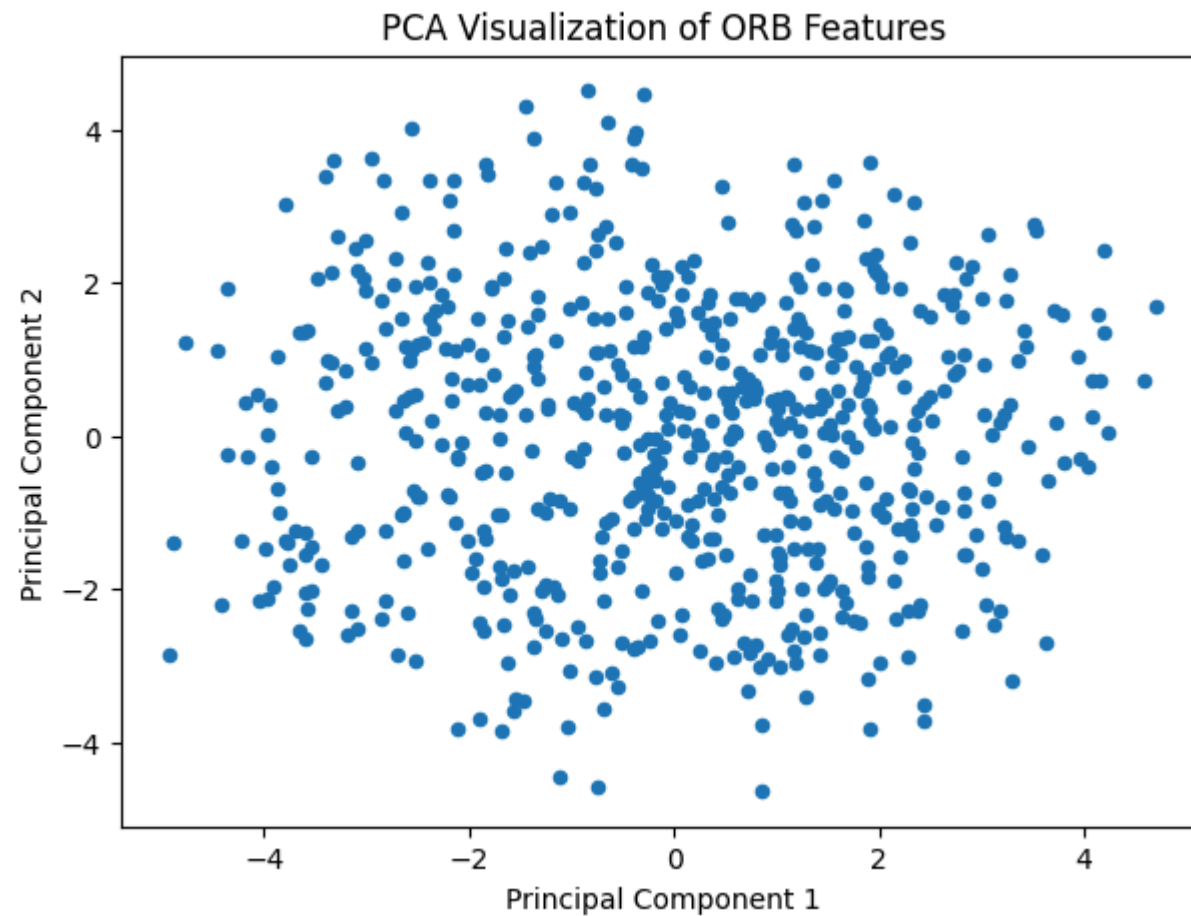
plt.subplot(1, 2, 2)
plt.imshow(feature_img2)
plt.title("Detected Features - Image 2")
plt.axis("off")

plt.show()
```

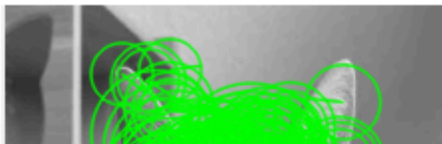
```
# -----  
# 8. Display Feature Matching  
# -----  
  
plt.figure(figsize=(14, 7))  
  
plt.imshow(matched_image)  
plt.title("Feature Matching using ORB + BFMatcher")  
plt.axis("off")  
  
plt.show()  
  
# -----  
# 9. Performance Analysis  
# -----  
  
end_time = time.time()  
  
processing_time = end_time - start_time  
  
print("\n----- PERFORMANCE ANALYSIS -----")  
print("Number of Features Image 1:", len(kp1))  
print("Number of Features Image 2:", len(kp2))  
print("Number of Matches:", len(matches))  
print("Best Matches Displayed:", len(best_matches))  
print("Processing Time:", round(processing_time, 4), "seconds")  
  
print("\n----- SYSTEM CONCLUSION -----")  
print("Preprocessing reduces image noise.")  
print("ORB detects important image features.")  
print("BFMatcher finds corresponding features.")  
print("PCA reduces descriptor dimensionality.")  
print("The complete pipeline provides a compact feature representation.")
```



```
Total Matches: 638  
Original Descriptor Dimensions: 32  
Reduced Dimensions: 2  
Explained Variance Ratio:  
[0.133071  0.10133303]  
Total Variance Preserved: 0.23440403
```



Detected Features - Image 1



Detected Features - Image 2

